mybatis是如何防止SQL注入的 - 淼淼之森 - 博客园

淼淼之森关注 - 12 粉丝 - 223 +加关注

阅读正文：

1、首先看一下下面两个sql语句的区别：

```
<select id="selectByNameAndPassword" parameterType="java.util.Map" resultMap="BaseResultMap">
select id, username, password, role
from user
where username = #{username,jdbcType=VARCHAR}
and password = #{password,jdbcType=VARCHAR}
</select>
```

```
<select id="selectByNameAndPassword" parameterType="java.util.Map" resultMap="BaseResultMap">
select id, username, password, role
from user
where username = ${username,jdbcType=VARCHAR}
and password = ${password,jdbcType=VARCHAR}
</select>
```

mybatis中的#和$的区别：

1、#将传入的数据都当成一个字符串，会对自动传入的数据加一个双引号。
如：where username=#{username}，如果传入的值是111,那么解析成sql时的值为where username="111", 如果传入的值是id，则解析成的sql为where username="id".
2、$将传入的数据直接显示生成在sql中。
如：where username=${username}，如果传入的值是111,那么解析成sql时的值为where username=111；
如果传入的值是;drop table user;，则解析成的sql为：select id, username, password, role from user where username=;drop table user;
3、#方式能够很大程度防止sql注入，$方式无法防止Sql注入。
4、$方式一般用于传入数据库对象，例如传入表名.
5、一般能用#的就别用$，若不得不使用“${xxx}”这样的参数，要手工地做好过滤工作，来防止sql注入攻击。
6、在MyBatis中，“${xxx}”这样格式的参数会直接参与SQL编译，从而不能避免注入攻击。但涉及到动态表名和列名时，只能使用“${xxx}”这样的参数格式。所以，这样的参数需要我们在代码中手工进行处理来防止注入。
【结论】在编写MyBatis的映射语句时，尽量采用“#{xxx}”这样的格式。若不得不使用“${xxx}”这样的参数，要手工地做好过滤工作，来防止SQL注入攻击。

2、什么是sql注入

sql注入解释：是一种代码注入技术，用于攻击数据驱动的应用，恶意的SQL语句被插入到执行的实体字段中（例如，为了转储数据库内容给攻击者）

SQL注入，大家都不陌生，是一种常见的攻击方式。**攻击者**在界面的表单信息或URL上输入一些奇怪的SQL片段（例如“or ‘1’=’1’”这样的语句），有可能入侵**参数检验不足**的应用程序。所以，在我们的应用中需要做一些工作，来防备这样的攻击方式。在一些安全性要求很高的应用中（比如银行软件），经常使用将**SQL语句**全部替换为**存储过程**这样的方式，来防止SQL注入。这当然是**一种很安全的方式**，但我们平时开发中，可能不需要这种死板的方式。

3、mybatis是如何做到防止sql注入的

MyBatis框架作为一款半自动化的持久层框架，其SQL语句都要我们自己手动编写，这个时候当然需要防止SQL注入。其实，MyBatis的SQL是一个具有“**输入+输出**”的功能，类似于函数的结构，参考上面的两个例子。其中，parameterType表示了输入的参数类型，resultType表示了输出的参数类型。回应上文，如果我们想防止SQL注入，理所当然地要在输入参数上下功夫。上面代码中使用#的即输入参数在SQL中拼接的部分，传入参数后，打印出执行的SQL语句，会看到SQL是这样的：

```
select id, username, password, role from user where username=? and password=?
```

不管输入什么参数，打印出的SQL都是这样的。这是因为MyBatis启用了预编译功能，在SQL执行前，会先将上面的SQL发送给数据库进行编译；执行时，直接使用编译好的SQL，替换占位符“?”就可以了。因为SQL注入只能对编译过程起作用，所以这样的方式就很好地避免了SQL注入的问题。

【底层实现原理】MyBatis是如何做到SQL预编译的呢？其实在框架底层，是JDBC中的PreparedStatement类在起作用，PreparedStatement是我们很熟悉的Statement的子类，它的对象包含了编译好的SQL语句。这种“准备好”的方式不仅能提高安全性，而且在多次执行同一个SQL时，能够提高效率。原因是SQL已编译好，再次执行时无需再编译。

```
//安全的，预编译了的
Connection conn = getConn();//获得连接
String sql = "select id, username, password, role from user where id=?"; //执行sql前会预编译号该条语句
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, id);
ResultSet rs=pstmt.executeUpdate();
......
```

```
//不安全的，没进行预编译
private String getNameByUserId(String userId) {
    Connection conn = getConn();//获得连接
    String sql = "select id,username,password,role from user where id=" + id;
    //当id参数为"3;drop table user;"时，执行的sql语句如下：
    //select id,username,password,role from user where id=3; drop table user;
    PreparedStatement pstmt =  conn.prepareStatement(sql);
    ResultSet rs=pstmt.executeUpdate();
    ......
}
```

【结论：】

#{}：相当于JDBC中的PreparedStatement
${}：是输出变量的值

简单说，#{}是经过预编译的，是安全的；${}是未经过预编译的，仅仅是取变量的值，是非安全的，存在SQL注入。
如果我们order by语句后用了${}，那么不做任何处理的时候是存在SQL注入危险的。你说怎么防止，那我只能悲惨的告诉你，你得手动处理过滤一下输入的内容。如判断一下输入的参数的长度是否正常（注入语句一般很长），更精确的过滤则可以查询一下输入的参数是否在预期的参数集合中。

4、参考文章

http://blog.csdn.net/yizhenn/article/details/52384601

https://www.cnblogs.com/200911/p/5869097.html

http://www.jb51.net/article/95314.htm

作者：淼淼之森
欢迎任何形式的转载，但请务必注明出处。
如果你觉得本文还可以，那就点击一下推荐，让更多人看到吧！
限于本人水平，如果文章和代码有表述不当之处，还请不吝赐教。